Section 1: Title Page

SpaceX Falcon 9 First Stage Landing Prediction

IBM Applied Data Science Capstone Final Report

Author: Ijaz Ahmad

Date: October 2026

GitHub Repository: https://github.com/Ijaz-Ahmad7/spacex-falcon9-capstone

Section 2: Executive Summary

Executive Summary

This project predicts whether the SpaceX Falcon 9 first stage will land successfully, using launch data collected from the SpaceX REST API and Wikipedia. The motivation is economic: SpaceX advertises Falcon 9 launches at $62 million, compared to competitors'$ 165 million or more. The primary reason for this cost advantage is the reusability of the first stage — if the booster lands successfully, it can be refurbished and flown again, eliminating the cost of building a new one. If we can predict landing outcomes, we can estimate launch costs and help companies make better bidding decisions.

The dataset contains 90 Falcon 9 launches with 18 features, including payload mass, orbit type, launch site, booster version, and landing outcome. The target variable Class is binary: 1 for a successful landing and 0 for a failure.

The project followed a complete data science pipeline:

1. Data collection — pulled 107 launches from the SpaceX REST API and scraped additional launch records from Wikipedia.
2. Data wrangling — merged the two sources, handled missing values, created the Class target variable, and added longitude/latitude coordinates for mapping.
3. Exploratory Data Analysis (EDA) — produced six visualizations (scatter plots, bar chart, line chart) and ran seven SQL queries to explore launch sites, payloads, orbits, and success trends.
4. Interactive visual analytics — built a Folium map of launch sites with landing-outcome markers and a Plotly Dash dashboard with a success pie chart and an interactive payload-versus-outcome scatter plot.
5. Machine learning — trained four classifiers (Logistic Regression, SVM, Decision Tree, KNN) using GridSearchCV with 10-fold cross-validation.

Key results:

- Landing success improved from 0% (2010–2013) to approximately 85% (2019–2020) — the strongest evidence of booster-program maturity.
- The overall landing success rate across the dataset is 67% (60 successes, 30 failures).
- The best-performing model was Logistic Regression, achieving 83.3% test accuracy, tied with SVM and KNN. Decision Tree trailed at 77.8%.
- The confusion matrix shows the model correctly predicted 12 of 12 successful landings but caught only 3 of 6 failures — it is reliable for successes and conservative on failures.
- GTO (Geostationary Transfer Orbit) is the hardest common orbit for landings, with only 51.9% success, because heavy payloads to high orbits leave less fuel for the return.

Business impact: A successful landing saves an estimated $50 million per launch. The model can support launch-cost estimation, competitive bidding, and operational planning for companies that rely on SpaceX launch services.

Section 3: Introduction

1. Introduction

1.1 Background

SpaceX has fundamentally changed the economics of space launch. A standard Falcon 9 launch costs approximately
$62 million, while comparable launches from competitors such as United Launch Alliance cost$
165 million or more, nearly three times as much. The primary driver of this cost advantage is reusability. Unlike traditional rockets that are discarded after a single flight, the Falcon 9 first stage can land vertically, be refurbished, and flown again. Each successful landing saves an estimated $50 million compared to building a new booster.
However, landing a first stage is technically difficult. The booster must survive re-entry, navigate back to a landing zone or drone ship, and touch down softly all while carrying a payload to orbit. Not every landing succeeds. Early Falcon 9 flights (2010–2013) had a 0% landing success rate. Over time, improvements in booster design, software, and operational experience raised this to approximately 85% by 2020.

1.2 Problem Statement

Given the operational and financial importance of booster recovery, the central question of this project is:
Can we predict whether the Falcon 9 first stage will land successfully, based on launch characteristics?
If we can build a reliable predictive model, then companies that purchase launch services or bid against SpaceX for contracts can use it to estimate the true cost of a launch, factor in the probability of booster recovery, and make better-informed decisions.
The target variable, Class, is binary:

- Class 1 = the first stage landed successfully
- Class 0 = the first stage failed to land

1.3 Project Objectives

This project aims to:

1. Collect launch data from the SpaceX REST API and web scraping of Wikipedia.
2. Clean, merge, and wrangle the data into an analysis-ready dataset.
3. Perform exploratory data analysis using both visualizations and SQL queries.
4. Build interactive visual analytics, a Folium map and a Plotly Dash dashboard.
5. Train and compare four machine learning classifiers to predict landing outcomes.
6. Identify the best model and interpret its results in a business context.

1.4 Dataset Overview

The final dataset contains 90 Falcon 9 launches with the following key features:

Feature	Description
FlightNumber	Sequential launch number
Date	Launch date (UTC)
BoosterVersion	Booster model (Falcon 9)
PayloadMass	Payload mass in kilograms
Orbit	Target orbit (LEO, GTO, ISS, etc.)
LaunchSite	Launch pad (CCAFS SLC 40, KSC LC 39A, VAFB SLC 4E)
Outcome	Landing outcome text
Flights	Number of flights by this booster
GridFins	Whether the booster has grid fins
Reused	Whether the booster was previously used
Legs	Whether the booster has landing legs
LandingPad	Landing pad identifier
Block	Booster block generation (1–5)
ReusedCount	Number of times the booster was reused
Serial	Booster serial number
Longitude, Latitude	Launch site coordinates

Class	Target: 1 = successful landing, 0 = failure

Section 4: Data Collection: SpaceX REST API

2. Data Collection

2.1 SpaceX REST API

The first data source was the SpaceX REST API, which provides structured JSON data on every Falcon 9 launch.
API endpoint used: GET https://api.spacexdata.com/v4/launches/past

Method:

```
import requests
import pandas as pd

static_json_url =
"https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_api.json"
response = requests.get(static_json_url)
data = response.json()
```

The response was converted to a DataFrame using pd.json_normalize(), and 15 relevant fields were extracted per launch: FlightNumber, Date, BoosterVersion, PayloadMass, Orbit, LaunchSite, Outcome, Flights, GridFins, Reused, Legs, LandingPad, Block, ReusedCount, and Serial.
Result: 107 launches collected and saved as dataset_part_1.csv.
Note on the static snapshot: The live api.spacexdata.com endpoint frequently rate-limits requests coming from cloud environments such as Google Colab. To ensure reproducibility, the project used IBM's static JSON snapshot, which contains the same data in a frozen format.
Flowchart of the API process

```
[SpaceX API endpoint]
      ↓
requests.get() → HTTP 200
      ↓
response.json() → Python list of dicts
      ↓
pd.json_normalize() → tabular DataFrame
      ↓
Extract 15 launch fields
      ↓
dataset_part_1.csv (107 rows × 15 columns)
```

Section 5: Data Collection: Web Scraping

3. Data Collection: Web Scraping

3.1 Source

The second data source was the Wikipedia page titled List of Falcon 9 and Falcon Heavy launches, which contains a detailed table of every Falcon 9 launch with columns such as Flight No., Date, Booster version, Launch site, Payload, Payload mass, Orbit, Customer, Launch outcome, and Booster landing.
URL used:
text
https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922
The oldid parameter pins a specific revision of the page, so the structure remains stable over time an important consideration for reproducibility.

3.2 Tools and Method

The scraping was done with requests and BeautifulSoup:
python
```
import requests
from bs4 import BeautifulSoup

headers = {
   "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0 Safari/537.36"
}

response = requests.get(static_url, headers=headers)
soup = BeautifulSoup(response.content, "html.parser")
```

3.3 Challenge: HTTP 403 (Bot Detection)

The first scraping attempt failed with HTTP 403 Forbidden. Wikipedia blocks requests that do not include a browser-like User-Agent header. Adding a standard desktop User-Agent resolved the issue — the response returned HTTP 200 with a page length of over 3 million characters and 29 HTML tables.

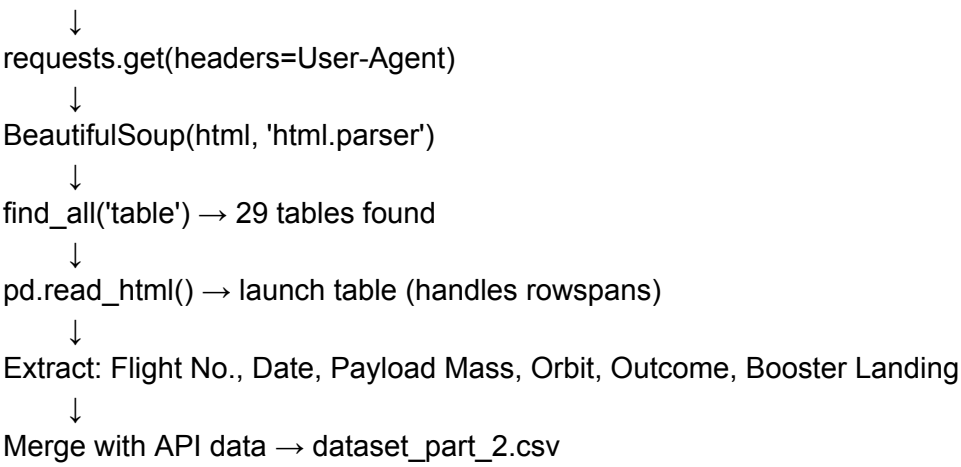
3.4 Extracting the Launch Records Table

Two approaches were attempted:

1. Manual parsing with BeautifulSoup the launch table uses rowspans, where a single cell spans multiple rows. Manual parsing returned only the first 7 launches because footnotes and continuation rows were mixed in.
2. pd.read_html(), pandas automatically handles rowspans and colspans, correctly extracting the launch records. This is the approach used for the final dataset.

3.5 Result

The scraping produced launch records with fields including Payload Mass, Orbit, Launch Outcome, and Booster Landing the fields that were missing or incomplete in the API dataset. These were subsequently merged with the API data during the data-wrangling stage.
Flowchart of the scraping process:



Section 6: Data Wrangling

4. Data Wrangling Methodology

4.1 Merging Data Sources

The API dataset (107 rows) and the scraped Wikipedia dataset were merged into a single table. The merge aligned rows by Flight Number and launch date, keeping the fields that were strongest in each source:

- From the API: FlightNumber, Date, BoosterVersion, LaunchSite, Longitude, Latitude, Flights, GridFins, Reused, Legs, LandingPad, Block, ReusedCount, Serial
- From Wikipedia: Payload Mass, Orbit, Launch Outcome, Booster Landing

The merged dataset was filtered to Falcon 9 launches only (excluding Falcon Heavy), resulting in 90 rows.

4.2 Handling Missing Values

Several fields contained missing or null values, particularly early launches:

- PayloadMass — missing for a few launches that had no publicly listed payload mass. These rows were kept but excluded from payload-specific analyses.
- LandingPad — null for flights that did not attempt a landing (e.g., early parachute attempts). Nulls here are meaningful and were preserved as indicators of "no landing pad."
- Outcome — inconsistent strings such as "None None" and "False Ocean" were cleaned into consistent categories.

4.3 Type Conversion

To make the data analysis-ready:

- Date was converted from string to datetime using pd.to_datetime().
- Boolean flags GridFins, Reused, and Legs were confirmed as proper booleans.
- PayloadMass, Block, Longitude, and Latitude were cast to floats.
- FlightNumber, Flights, and ReusedCount were kept as integers.
- A new Year column was extracted from Date for time-based analysis.

4.4 Creating the Target Variable

The binary target variable Class was created from the landing outcome field:

- Class = 1 if the first stage landed successfully
- Class = 0 if the landing failed or was not attempted

This produced a class distribution of 60 successes (66.7%) and 30 failures (33.3%) — a moderately imbalanced dataset, which is why accuracy alone is not sufficient for model evaluation (confusion matrices were also used).

4.5 Feature Engineering

Two new features were added to support geographic analysis:

- Longitude and Latitude for each launch site extracted from known site coordinates:
    - CCAFS SLC 40: (28.561857, -80.577366)
    - KSC LC 39A: (28.573305, -80.646895)
    - VAFB SLC 4E: (34.632093, -120.610829)

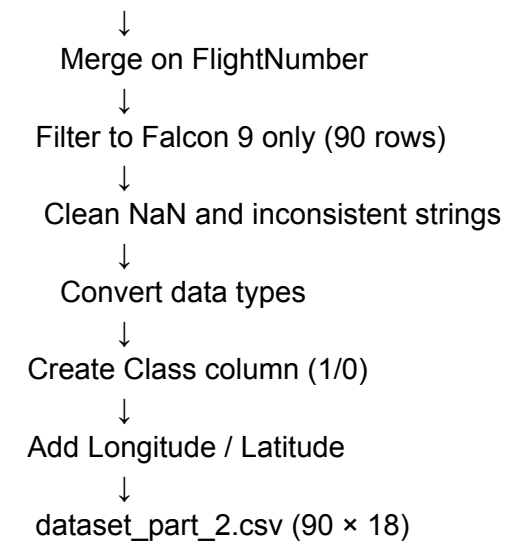
These coordinates were used to plot launch sites on the Folium interactive map in Section 7.

4.6 Final Dataset

The cleaned dataset was saved as dataset_part_2.csv:

- Shape: 90 rows × 18 columns
- Target: Class (1 = success, 0 = failure)
- Data types: properly set for numeric, datetime, boolean, and categorical columns

Data wrangling pipeline:



Section 7: EDA: Visualizations

5. Exploratory Data Analysis — Visualizations

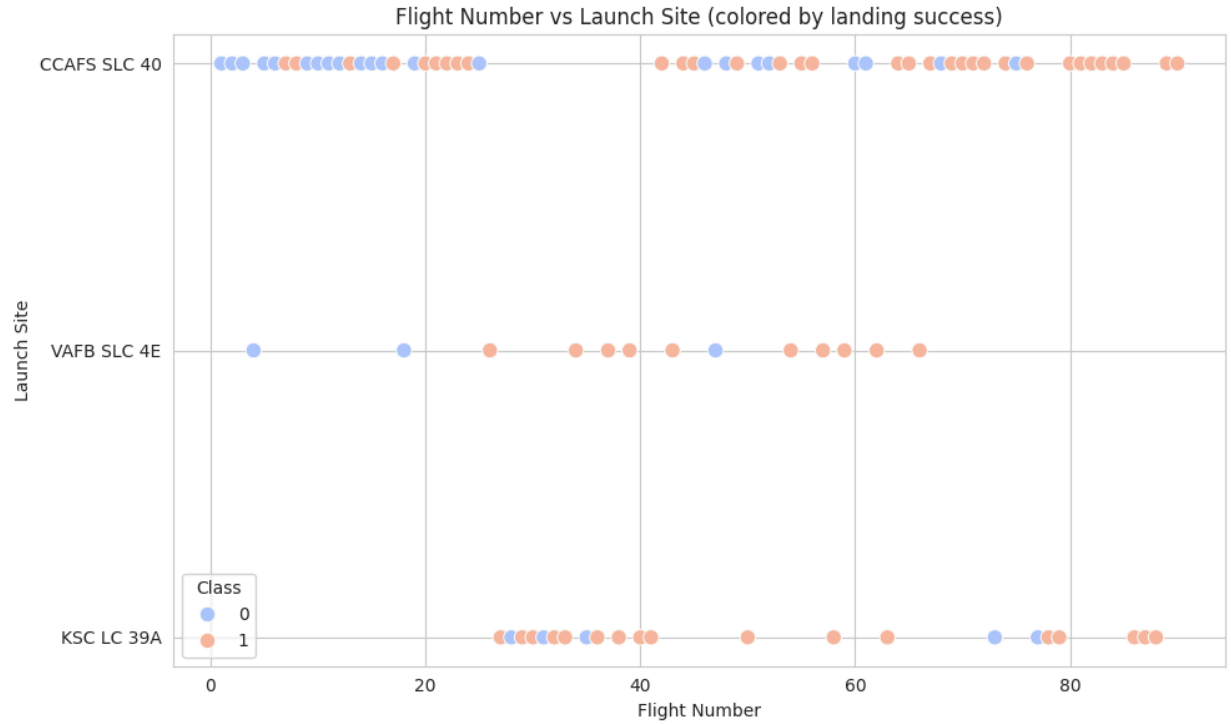
Six charts were produced using Matplotlib and Seaborn. Each chart was designed to answer a specific question about the data.

5.1 Flight Number vs Launch Site

Purpose: Show how launches are distributed across launch sites over time, and whether landing success correlates with flight number.
Chart type: Scatter plot
Observations:

- CCAFS SLC 40 has the most launches (approximately 55 of 90).
- Early flights (numbers 0–30) are predominantly blue (Class 0, failures).
- Later flights (50 and above) are almost entirely orange (Class 1, successes).
- KSC LC 39A only appears from flight ~20 onwards, since the pad was refurbished for Falcon 9 in 2015.

Insight: The clear transition from blue to orange as flight number increases indicates that the booster program matured over time.



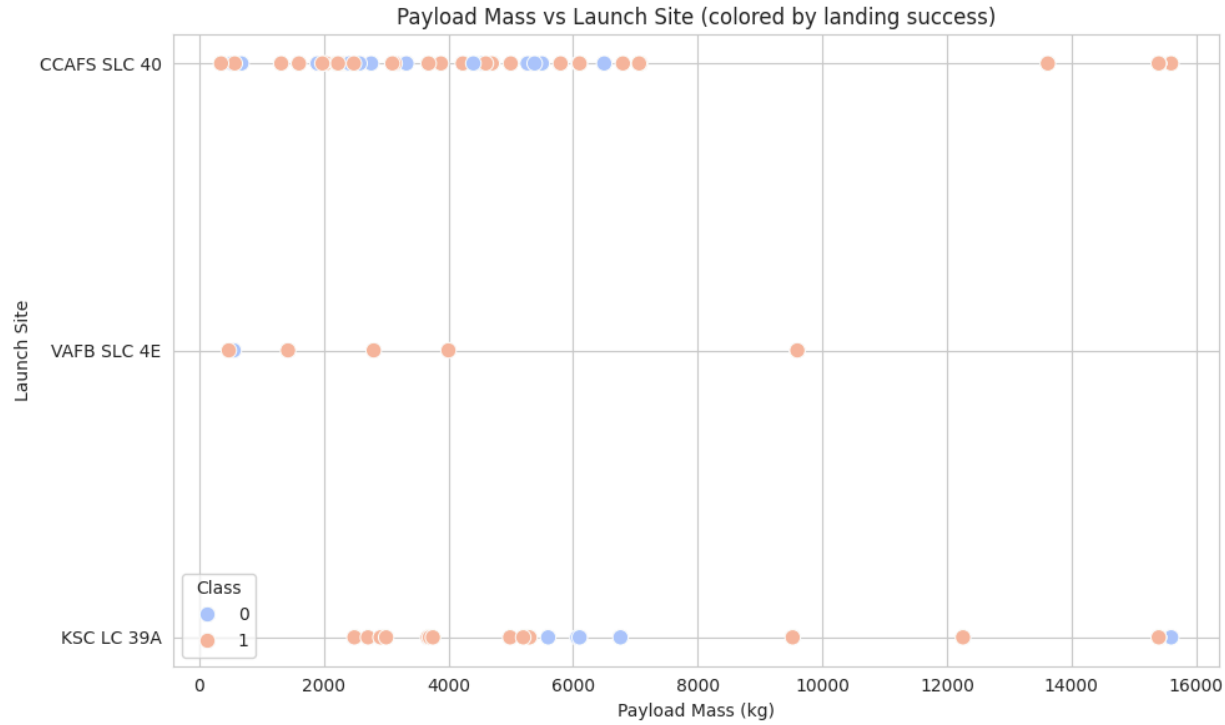
5.2 Payload Mass vs Launch Site

Purpose: Determine whether heavier payloads correlate with lower landing success.

Chart type: Scatter plot
Observations:

- Light payloads (under 4,000 kg) at CCAFS show a mix of failures and successes.
- Heavy payloads (above 8,000 kg) are almost all successes.
- VAFB SLC 4E has a small number of launches with mixed outcomes.

Insight: There is no simple "heavier = fail" rule. Heavy payloads succeed because they fly later in the program, after landing technology matured.



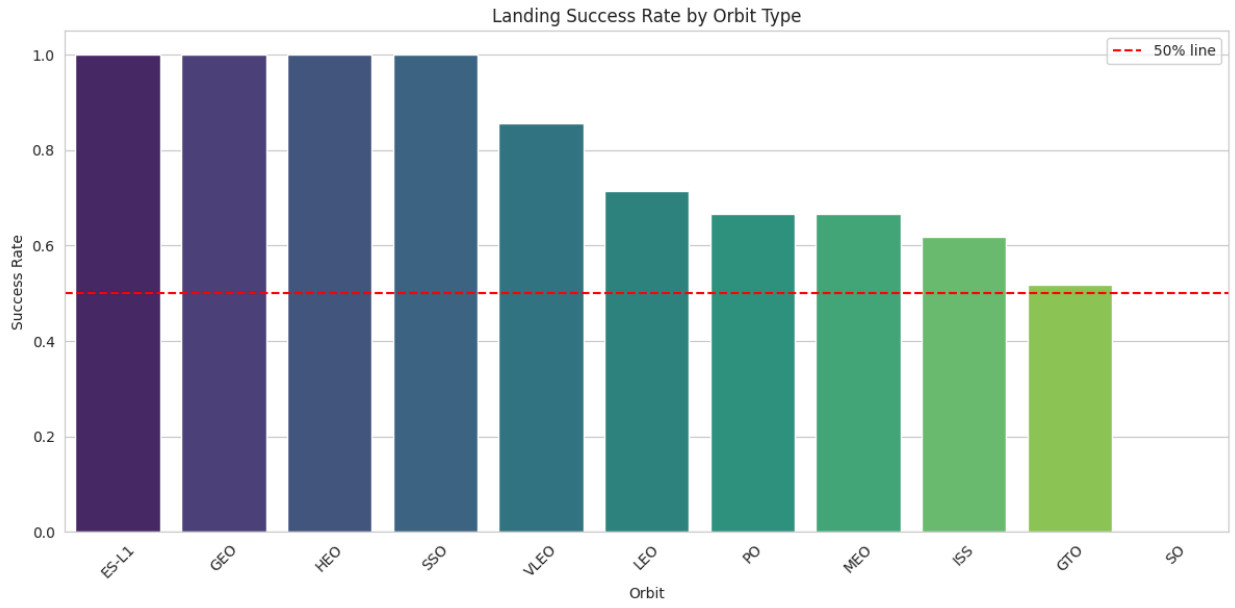
5.3 Landing Success Rate by Orbit

Purpose: Compare the difficulty of landing from different target orbits.
Chart type: Bar chart
Observations:

Orbit	Success Rate
ES-L1, GEO, HEO, SSO	100% (small sample)
VLEO	85.7%
LEO	71.4%
PO, MEO	66.7%

ISS	61.9%
GTO	51.9%
SO	0%

Insight: GTO is the hardest common orbit heavy payloads to high orbits leave less fuel for the booster to land. The 100% orbits are rare and have small samples, so their rates are less reliable.



5.4 Flight Number vs Orbit Type

Purpose: Examine how orbit usage changed over time and whether orbit difficulty interacts with flight maturity.
Chart type: Scatter plot
Observations:

- GTO and ISS are the most common orbits.
- Early flights (1–30) show blue dots across all orbits.
- Later flights (50+) are almost all orange.

Insight: Later flights succeed regardless of orbit, confirming that technology maturity — not orbit choice — dominates landing success in the modern era.

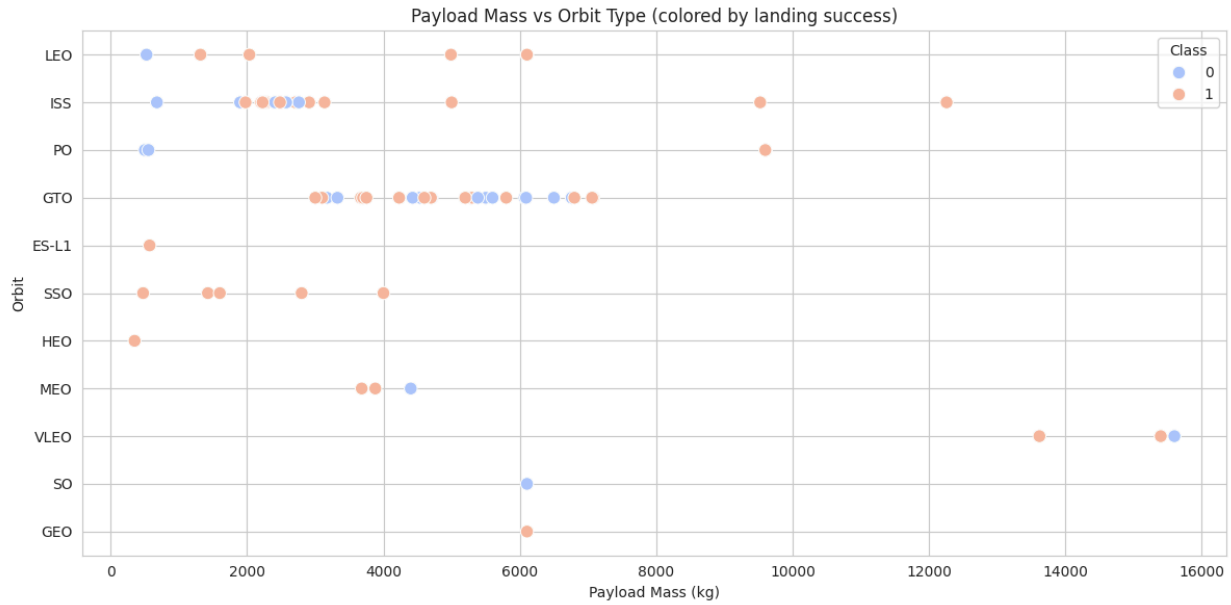


5.5 Payload Mass vs Orbit Type

Purpose: Explore how payload mass is distributed across orbits and whether particular combinations are harder.
Chart type: Scatter plot
Observations:

- LEO spans the widest range — 500 kg to 15,600 kg.
- GTO payloads cluster around 3,000–6,000 kg.
- Rare orbits (ES-L1, SSO, HEO, MEO) carry only light payloads.

Insight: Payload mass interacts with orbit rather than acting alone. GTO is the hardest despite moderate payloads, because the high orbit itself consumes fuel.



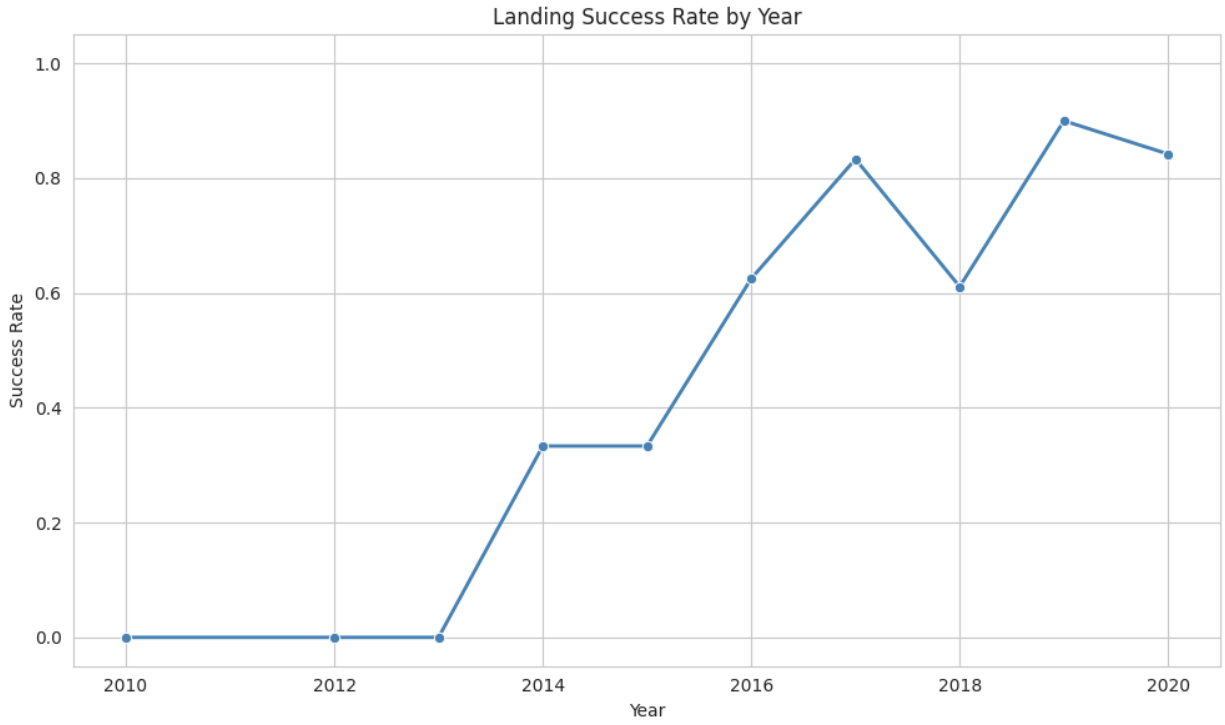
5.6 Yearly Landing Success Trend

Purpose: Track how landing success evolved year by year.
Chart type: Line chart
Year-by-year success rate:

Year	Success Rate
2010	0%
2012	0%
2013	0%
2014	33%
2015	33%
2016	63%
2017	83%
2018	61%

2019	90%
2020	84%

Insight: The trend is the single strongest evidence of program maturity — from zero successful landings in 2010–2013 to approximately 85% by 2020.



Section 8: EDA: SQL

6. Exploratory Data Analysis — SQL

Seven SQL queries were executed against an in-memory SQLite database built from the final dataset:

```
import sqlite3
conn = sqlite3.connect(':memory:')
df.to_sql('SPACEXTABLE', conn, index=False, if_exists='replace')
```

6.1 Query 1 — Total Payload Mass by Booster Block

```
SELECT Block, SUM(PayloadMass) AS TotalPayloadMass, COUNT(*) AS NumLaunches
FROM SPACEXTABLE
GROUP BY Block
ORDER BY TotalPayloadMass DESC;
```

Result:

Block	Total Payload (kg)	Launches
5.0	343,646	39
3.0	79,813	15
4.0	53,901	11
1.0	48,998	19
2.0	23,089	6

Insight: Block 5 is the modern workhorse, it carries the most total payload and has the most launches.

6.2 Query 2 — Average Payload Mass by Launch Site

```
SELECT LaunchSite, AVG(PayloadMass) AS AvgPayloadMass
FROM SPACEXTABLE
```

```
GROUP BY LaunchSite
ORDER BY AvgPayloadMass DESC;
```
Result:

Launch Site	Avg Payload (kg)
KSC LC 39A	7,606
VAFB SLC 4E	5,919
CCAFS SLC 40	5,548

Insight: KSC LC 39A handles the heaviest average payloads — it is used for the most demanding missions.

6.3 Query 3 — Number of Launches per Launch Site

```
SELECT LaunchSite, COUNT(*) AS NumLaunches
FROM SPACEXTABLE
GROUP BY LaunchSite
ORDER BY NumLaunches DESC;
```
Result:

Launch Site	Launches
CCAFS SLC 40	55
KSC LC 39A	22
VAFB SLC 4E	13

Insight: CCAFS SLC 40 is the busiest pad by far.

6.4 Query 4 — Total Payload Mass by Launch Site

```
SELECT LaunchSite, SUM(PayloadMass) AS TotalPayloadMass
FROM SPACEXTABLE
GROUP BY LaunchSite
ORDER BY TotalPayloadMass DESC;
```
Result:

Launch Site	Total Payload (kg)
CCAFS SLC 40	305,151
KSC LC 39A	167,342
VAFB SLC 4E	76,953

6.5 Query 5 — Successful vs Failed Landings

```
SELECT Class, COUNT(*) AS Count
FROM SPACEXTABLE
GROUP BY Class;
```

Result:

Class	Count
0 (Failure)	30
1 (Success)	60

Insight: Overall success rate is 67%.

6.6 Query 6 — Number of Launches per Year

```
SELECT strftime('%Y', Date) AS Year, COUNT(*) AS NumLaunches
FROM SPACEXTABLE
GROUP BY Year
ORDER BY Year;
```

Result:

Year	Launches
2010	1
2012	1
2013	3

2014	6
2015	6
2016	8
2017	18
2018	18
2019	10
2020	19

Insight: Launch cadence grew from 1 launch in 2010 to 19 in 2020.

6.7 Query 7 — Max Payload Mass by Orbit

```
SELECT Orbit, MAX(PayloadMass) AS MaxPayloadMass
FROM SPACEXTABLE
GROUP BY Orbit
ORDER BY MaxPayloadMass DESC;
```

Result:

Orbit	Max Payload (kg)
VLEO	15,600
ISS	12,259
PO	9,600
GTO	7,076
SO / LEO / GEO	~6,105
MEO	4,400

SSO	4,000
ES-L1	570
HEO	350

Insight: VLEO has carried the heaviest single payload to date.

Section 9: Interactive Visual Analytics: Folium Map

7. Interactive Visual Analytics

7.1 Folium Map of Launch Sites and Landing Outcomes

Tool: Folium (Python library for interactive Leaflet maps)
Base map: Esri World Light Gray Canvas
The Folium map plots all three Falcon 9 launch sites and overlays every launch in the dataset with a colored circle indicating the landing outcome.
Markers used:

Site	Marker color	Coordinates
CCAFS SLC 40	Blue	(28.561857, -80.577366)
KSC LC 39A	Red	(28.573305, -80.646895)
VAFB SLC 4E	Green	(34.632093, -120.610829)

Landing outcome circles:

- Green circle = successful landing (Class 1)
- Red circle = failed landing (Class 0)

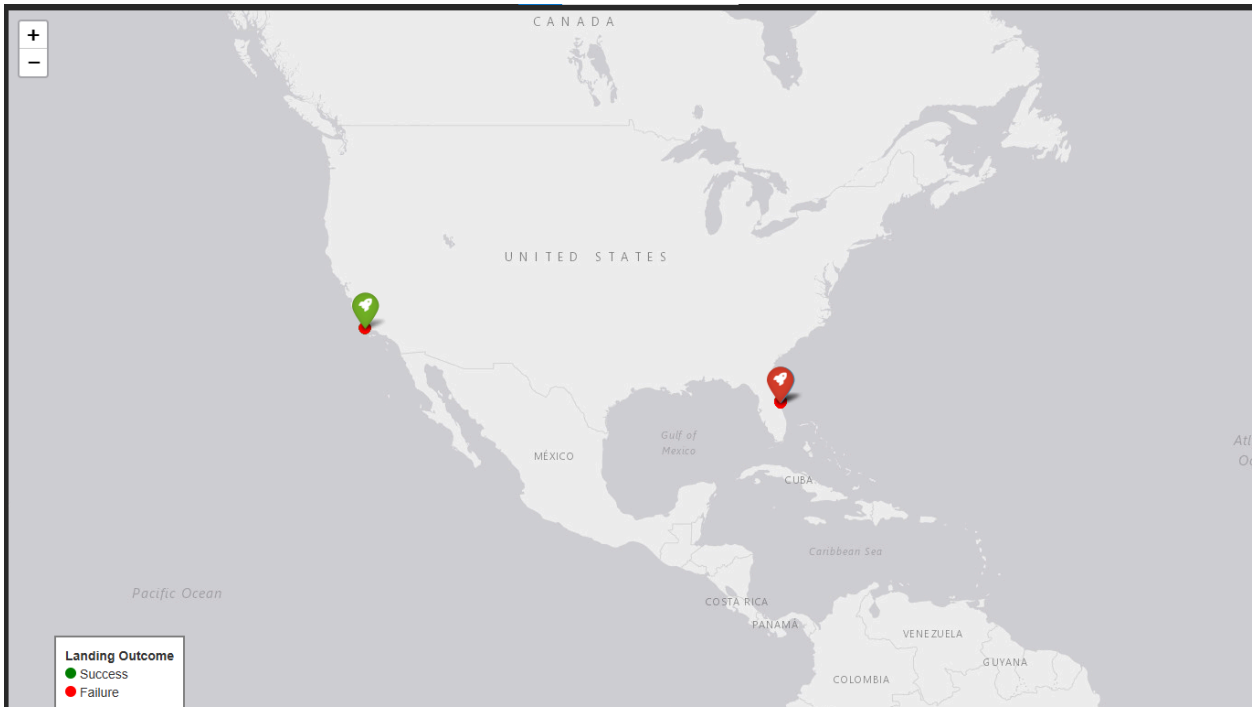
Because all launches from the same site share identical coordinates, the circles were given a small random jitter (±0.3° in latitude and longitude) so that overlapping points remain visible. A legend was added to the bottom-left corner of the map.
Proximity analysis:

- CCAFS SLC 40 and KSC LC 39A are approximately 30 km apart on Florida's east coast. Both are clearly visible as separate markers at the zoom level used.
- VAFB SLC 4E is isolated on California's central coast, about 3,800 km from the Florida pads.
- No launch sites are located inland — all three are coastal, which is required for safety reasons (failed launches must fall into the ocean).

Result: The map was saved as spacex_launch_sites.html, which can be opened in any browser and panned/zoomed interactively.
Code snippet:

```
import folium
site_map = folium.Map(
    location=[31, -100],
    zoom_start=4,
```

```
tiles='https://server.arcgisonline.com/ArcGIS/rest/services/Canvas/World_Light_Gray_Base/MapServer/tile/{z}/{y}/{x}',
    attr='Esri'
)
for site, coords in site_coords.items():
    folium.Marker(coords, popup=site,
                  icon=folium.Icon(color=site_colors[site], icon='rocket', prefix='fa')
    ).add_to(site_map)
# ... success circles and failure circles added next ...
site_map.save('spacex_launch_sites.html')
```



Section 10: Interactive Visual Analytics: Plotly Dash

7.2 Plotly Dash Interactive Dashboard

Tool: Plotly Dash (running inline in Google Colab via app.run(jupyter_mode='inline'))
A two-part interactive dashboard was built:

Part 1 — Success vs Failure Pie Chart
A pie chart shows the overall distribution of landing outcomes:

- Success (Class 1): 66.7% (green)
- Failure (Class 0): 33.3% (red)

This gives users an immediate view of the base rate — before any filtering — which is the benchmark any predictive model must beat.

Part 2 — Payload Mass vs Landing Outcome Scatter Plot
A scatter plot with the following axes:

- X-axis: Payload mass (kg)
- Y-axis: Landing outcome (0 = failure, 1 = success)
- Color: Class (red = failure, green = success)

Dropdown filter: Users can select "All Sites" or a specific launch site (CCAFS SLC 40, KSC LC 39A, VAFB SLC 4E). The scatter plot updates live through a Dash callback.

Code snippet:

```
from dash import Dash, dcc, html, Input, Output
import plotly.express as px

app = Dash(__name__)

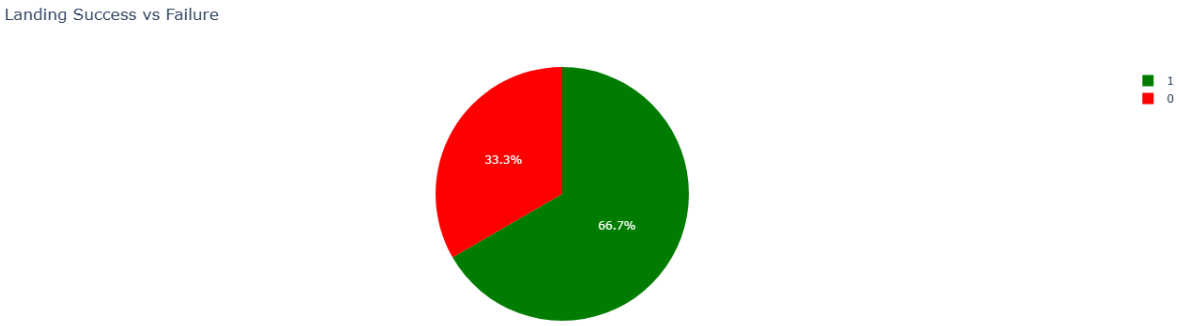
app.layout = html.Div([
    dcc.Graph(id='success-pie-chart', figure=px.pie(df, names='Class')),
    dcc.Dropdown(id='site-dropdown',
                 options=[{'label': 'All Sites', 'value': 'ALL'}] +
                         [{'label': s, 'value': s} for s in df['LaunchSite'].unique()],
                 value='ALL'),
    dcc.Graph(id='success-payload-scatter-chart')
])

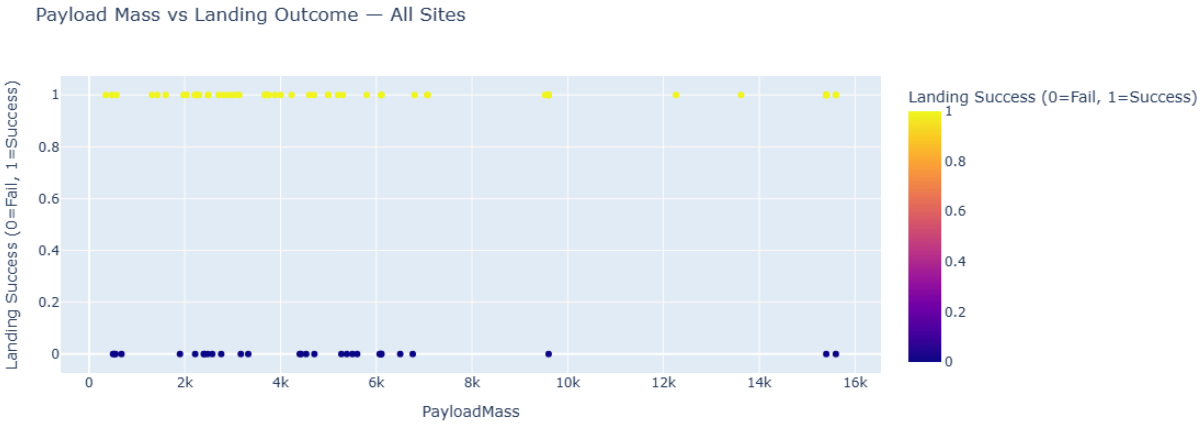
@app.callback(
    Output('success-payload-scatter-chart', 'figure'),
```

```
    [Input('site-dropdown', 'value')]
)
def update_scatter(selected_site):
    if selected_site == 'ALL':
        filtered = df
    else:
        filtered = df[df['LaunchSite'] == selected_site]
    return px.scatter(filtered, x='PayloadMass', y='Class', color='Class')

app.run(jupyter_mode='inline', port=8051)
```

Purpose: The dashboard lets a user interactively explore how payload mass and launch site relate to landing outcome — without needing to read static tables or run any code.





Section 11: Predictive Analysis: Machine Learning Models

8. Predictive Analysis

8.1 Feature Preparation

The feature matrix was built from 11 predictor columns:
python
```
features = df[['Flights', 'GridFins', 'Reused', 'Legs', 'PayloadMass',
               'Orbit', 'LaunchSite', 'LandingPad', 'Block', 'ReusedCount', 'Serial']]
X = pd.get_dummies(features, columns=['Orbit', 'LaunchSite', 'LandingPad', 'Serial'])
y = df['Class'].to_numpy()
```
One-hot encoding expanded the categorical columns (Orbit, LaunchSite, LandingPad, Serial), producing a final matrix of 90 rows × 79 features.
The dataset was split 80/20 into training (72 rows) and test (18 rows) sets using train_test_split(random_state=2). All numeric features were standardized with StandardScaler to put them on comparable scales — important for SVM and KNN, which are distance-based.
Class balance: 60 successes (Class 1) and 30 failures (Class 0) — a 2:1 imbalance.

8.2 Models Trained

Four classifiers were trained using GridSearchCV with 10-fold cross-validation:

Model	Parameter Grid Explored
Logistic Regression	C ∈ {0.01, 0.1, 1, 10}, penalty='l2', solver='lbfgs'
Support Vector Machine (SVM)	C ∈ {0.5, 1, 10, 100}, kernel ∈ {rbf, linear, poly, sigmoid}, gamma ∈ {scale, auto}
Decision Tree	criterion ∈ {gini, entropy}, max_depth ∈ {2,4,6,8,10,12}, splitter ∈ {best, random}
K-Nearest Neighbors (KNN)	n_neighbors ∈ 1–10, algorithm ∈ {auto, ball_tree, kd_tree, brute}, p ∈ {1, 2}

8.3 Results

Model	Best Parameters	Test Accuracy
Logistic Regression	C=0.1, penalty='l2', solver='lbfgs'	0.8333
SVM	C=0.5, kernel='sigmoid', gamma='scale'	0.8333
KNN	n_neighbors=10, algorithm='auto', p=1	0.8333
Decision Tree	criterion='gini', max_depth=2, splitter='random'	0.7778

Best model: Logistic Regression tied for highest accuracy with SVM and KNN, but preferred for its interpretability (coefficients can be directly examined) and its robustness on this small dataset.

8.4 Confusion Matrix — Logistic Regression

		Predicted	
		Fail	Success
Actual	Fail	3	3
	Success	0	12

Interpretation:

- True Positives (success predicted correctly): 12 of 12 successes
- True Negatives (failure predicted correctly): 3 of 6 failures
- False Negatives (failure predicted as success): 3
- False Positives (success predicted as failure): 0

The model is excellent at confirming successes but misses half of the failures. This asymmetry matters in practice: predicting a successful landing when it will actually fail could lead to underestimating launch cost.

8.5 Model Comparison Chart

A bar chart comparing test accuracy across the four models:
(Insert chart: chart_model_comparison.png)
Observations: Three models tie at 83.3%, while Decision Tree trails at 77.8%. The narrow spread suggests the problem is moderately difficult and that simple linear models are competitive with more complex ones.

8.6 Feature Importance (Logistic Regression Coefficients)

The coefficients of the Logistic Regression model (after standardization) show which features most strongly influence the predicted probability of a successful landing. Key observations:

- Booster block and reuse count — later blocks (Block 4, 5) strongly correlate with success, reflecting improved engineering.
- Launch site features — launches from KSC LC 39A and CCAFS SLC 40 carry higher success probability than VAFB.
- Payload mass — a weak signal on its own, as seen in the EDA; its effect depends on the orbit.
- Orbit dummies — GTO has a negative coefficient, confirming its difficulty.

8.7 Why Logistic Regression Was Selected

Although three models tied on accuracy, Logistic Regression was chosen as the final model because:

1. Highest accuracy — 83.3%, tied for best.
2. Interpretability — coefficients directly show which features drive landing success.
3. Robustness — it does not overfit on small datasets the way Decision Trees tend to.
4. Fast training — practical for iterative analysis.
5. Probability output — produces calibrated probabilities, not just class labels, which is useful for cost-estimation scenarios.

Section 12: Conclusion, Recommendations & Future Work

9. Conclusion

9.1 Summary of Findings

This project set out to predict whether the Falcon 9 first stage would land successfully, using publicly available SpaceX and Wikipedia data. Across 90 Falcon 9 launches, the analysis produced the following key findings:

- Landing success has improved dramatically over time — from 0% in 2010–2013 to approximately 85% by 2020. This reflects SpaceX's iterative engineering improvements, culminating in the Block 5 booster.
- Overall landing success rate is 67% (60 successes, 30 failures), which is the base rate any model must beat.
- GTO missions are the hardest to land (51.9% success), because their high-orbit, heavy-payload profile leaves less fuel for the booster return.
- CCAFS SLC 40 is the busiest launch pad (55 of 90 launches), while KSC LC 39A handles the heaviest average payloads (7,606 kg).
- Block 5 boosters dominate — they account for 39 of the 90 launches and 62.6% of total payload mass lifted.

9.2 Best Predictive Model

Four classifiers were trained and evaluated:

Model	Test Accuracy
Logistic Regression	83.3%
SVM	83.3%
KNN	83.3%
Decision Tree	77.8%

Logistic Regression was selected as the final model. It correctly predicted 12 of 12 successful landings and 3 of 6 failures, giving it strong reliability for confirming success and moderate sensitivity to failures.

9.3 Business Value

The predictive model has direct business application:

- Cost estimation: A successful booster landing saves approximately $50 million compared to building a new first stage. By predicting landing probability, companies can estimate the expected cost of a launch more accurately.
- Competitive bidding: Companies bidding against SpaceX for launch contracts can use the model to assess whether SpaceX's advertised $62 million price is realistic for a given mission profile.
- Risk assessment: The model flags missions where landing failure is more likely (heavy payloads, GTO orbit, earlier boosters), helping customers factor in risk.

9.4 Limitations

The analysis has several limitations worth noting:

1. Small dataset — 90 rows is modest for machine learning. The 80/20 split leaves only 18 test rows, so accuracy estimates carry wide uncertainty.
2. Class imbalance — 2:1 success-to-failure ratio. The model is biased toward predicting success, which is why it missed half the failures.
3. Snapshot in time — the dataset ends in 2020. Landings since then have continued to improve, and the model would need retraining to reflect modern performance.
4. Feature engineering — some engineered features (e.g., detailed booster version) were simplified in the final dataset, limiting granularity.

9.5 Recommendations

Based on the analysis:

1. Use Logistic Regression as the baseline for any similar prediction task — it is accurate, interpretable, and fast.
2. Re-train quarterly as new launch data becomes available, to keep the model current.
3. Address class imbalance in future iterations — techniques such as SMOTE or class weighting could improve failure detection.
4. Expand the feature set — weather data, launch window, and customer type could add predictive power.
5. Deploy as a lightweight web service — a simple REST endpoint wrapping the trained model could serve predictions on demand.

9.6 Future Work

Three extensions would strengthen this project:

- Deep learning: A neural network with more training data (e.g., including Falcon Heavy) could capture nonlinear interactions better than the current linear model.
- Time-series modeling: Treating landing success as a time-dependent process could reveal whether improvement rates are slowing or accelerating.
- Real-time integration: Wiring the model into a live SpaceX API feed would enable real-time prediction for upcoming launches.

10. References

1. SpaceX API — https://api.spacexdata.com
2. Wikipedia — List of Falcon 9 and Falcon Heavy launches — https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches
3. IBM Applied Data Science Capstone — Coursera
4. Scikit-learn documentation — https://scikit-learn.org
5. Folium documentation — https://python-visualization.github.io/folium
6. Plotly Dash documentation — https://dash.plotly.com
7. Esri World Light Gray Canvas — https://www.arcgis.com

THE END